NUTRITION APP

Documentación Técnica del Proyecto

Técnicas de Programación Proyecto #2

Luis Esteban Vega Varela

Versión 2.0 2026

Contenido

1. Introducción 4
1.1 Objetivos del proyecto 4
1.2 Tecnologías utilizadas 5
2. Decisiones de Diseño 6
2.1 Arquitectura MVC 6
2.2 Patron Repository 6
2.3 Herencia y Polimorfismo 6
2.4 Migración a Blazor Server 7
2.5 Base de datos SQLite con Entity Framework Core 7
2.6 Roles de usuario 7
2.7 Activación y desactivación de usuarios 7
2.8 Reportes CSV y PDF 7
3. Diagrama de Base de Datos 8
3.1 Descripción de tablas 8
4. Desarrollo 9
4.1 Estructura del proyecto 9
4.2 Requisitos para ejecutar el programa 9
4.3 Como ejecutar el programa 9
4.4 Credenciales de prueba 9
4.5 Flujo de la aplicación 10
5. Análisis de Resultados, Aprendizajes y Conclusiones 11
5.1 Análisis de resultados 11
5.2 Conclusiones 11

1. Introducción

El mundo actual exige que los profesionales en tecnología sean capaces de construir soluciones de software que no solo funcionen, sino que estén bien diseñadas, sean fáciles de mantener y puedan crecer con el tiempo. NutritionApp nace precisamente con ese enfoque: una aplicación desarrollada en C# cuyo propósito principal es ayudar a los usuarios a gestionar su alimentación diaria, monitorear su consumo nutricional y alcanzar sus objetivos de salud de manera organizada y personalizada.

El proyecto fue concebido y desarrollado en dos iteraciones bajo la metodología SCRUM. La primera iteración establecía las bases del sistema con Windows Forms como interfaz gráfica y archivos JSON como almacenamiento. La segunda iteración, documentada en este documento, representa una evolución significativa: se migro la interfaz a Blazor Server, se implementó SQLite como base de datos relacional, se agregó un rol de administrador con funcionalidades exclusivas, se incorporaron reportes descargables en CSV y PDF, y se desarrollaron pruebas unitarias para los controladores.

Desde el punto de vista técnico, la aplicación mantiene la arquitectura Modelo-Vista-Controlador (MVC) como columna vertebral del proyecto, garantizando una separación clara entre la lógica de negocio, el acceso a datos y la interfaz gráfica. El patrón Repository facilito la migración de JSON a SQLite sin modificar los controladores, validando así la efectividad de las decisiones arquitectónicas tomadas desde el inicio.

La presente documentación técnica describe en detalle las decisiones de diseño, el proceso de construcción del sistema, las instrucciones para ejecutar la aplicación y los aprendizajes obtenidos a lo largo de ambas iteraciones.

1.1 Objetivos del proyecto

- Aplicar la arquitectura MVC en una aplicación web
- Implementar herencia, polimorfismo y encapsulamiento con la jerarquía de planes nutricionales.
- Aplicar el patron Repository para el acceso a datos, facilitando la migración entre tecnologías.
- Migrar la interfaz de Windows Forms a Blazor Server.
- Implementar SQLite como base de datos relacional en sustitución de archivos JSON.
- Agregar un rol de administrador con funcionalidades exclusivas de gestión.
- Generar reportes descargables en CSV y PDF.

1.2 Tecnologías utilizadas

Lenguaje	C#
Framework	.NET 10.0
Interfaz grafica (v1)	Windows Forms
Interfaz grafica (v2)	Blazor Server
Base de datos	SQLite con Entity Framework Core
Almacenamiento (v1)	Archivos JSON con Newtonsoft.Json 13.0.3
Generacion de PDF	iText7 con bouncy-castle-adapter
IDE	Visual Studio Code / Visual Studio Community
Control de versiones	Git / GitHub
Gestion de proyecto	Jira (SCRUM)
Testing	MSTest

2. Decisiones de Diseño

2.1 Arquitectura MVC

Se mantuvo la arquitectura MVC en la segunda iteración. La migración de Windows Forms a Blazor solo afecto la capa de Vista, mientras que los Modelos, Controladores y Servicios permanecieron sin cambios. Esto demostró la solidez de la separación de responsabilidades implementada desde el inicio.

Capa	Responsabilidad	Tecnología v1	Tecnología v2
Model	Estructura de datos del sistema.	C# Classes	C# Classes + EF Core
View	Interfaz gráfica e interacción.	Windows Forms	Blazor Server (.razor)
Controller	Logica de negocio.	C# Classes	C# Classes (sin cambios)

2.2 Patron Repository

El patron Repository fue la decisión más importante del proyecto. Al centralizar el acceso a datos en clases dedicadas, la migración de JSON a SQLite se realizó modificando únicamente los repositorios, sin tocar controladores ni vistas. Esto valida la efectividad del patrón en proyectos reales.

UserRepository	Lee y guarda usuarios en la base de datos.
MenuRepository	Lee y guarda menús con sus items.
FoodRepository	Lee y guarda el catálogo de alimentos.

2.3 Herencia y Polimorfismo

Se mantuvo la jerarquía de clases NutritionPlan en la segunda iteración. La clase abstracta define los métodos que todas las dietas deben implementar, y cada subclase calcula sus propios macronutrientes.

Clase	Tipo	Proteína	Carbohidratos	Grasas
NutritionPlan	Abstracta (base)	-	-	-
StandardPlan	Concreta	30%	40%	30%
KetoPlan	Concreta	25%	5%	70%
VegetarianPlan	Concreta	20%	55%	25%

2.4 Migración a Blazor Server

Se eligió Blazor Server por su facilidad de aprendizaje y compatibilidad con C#. La capa de Vista fue completamente reescrita en componentes. razor, manteniendo la misma lógica de negocio de los controladores. La navegación entre paginas se maneja con NavigationManager y el estado del usuario autenticado se mantiene con un SessionService registrado como Singleton.

2.5 Base de datos SQLite con Entity Framework Core

Se eligió SQLite por su simplicidad para un proyecto universitario. Entity Framework Core permite definir las tablas como clases C# y genera la base de datos automáticamente. El DbInitializer carga los datos de los archivos JSON al crear la base de datos por primera vez, garantizando que el sistema tenga datos de prueba desde el inicio.

2.6 Roles de usuario

Se agrego el campo Role al modelo User para distinguir entre usuarios estándar y administradores. El sistema redirige al usuario según su rol al hacer login: los usuarios estándar van al Dashboard y los administradores van al Admin Panel.

User	Acceso al Dashboard, menús, estadísticas, perfil y reportes propios.
Admin	Acceso al panel administrativo: gestión de usuarios, productos y estadísticas globales.

2.7 Activación y desactivación de usuarios

Se agrego el campo IsActive al modelo User para permitir al administrador desactivar cuentas sin eliminarlas. Un usuario desactivado no puede hacer login aunque sus datos permanecen en el sistema. El administrador puede reactivar la cuenta en cualquier momento desde el panel de gestion de usuarios.

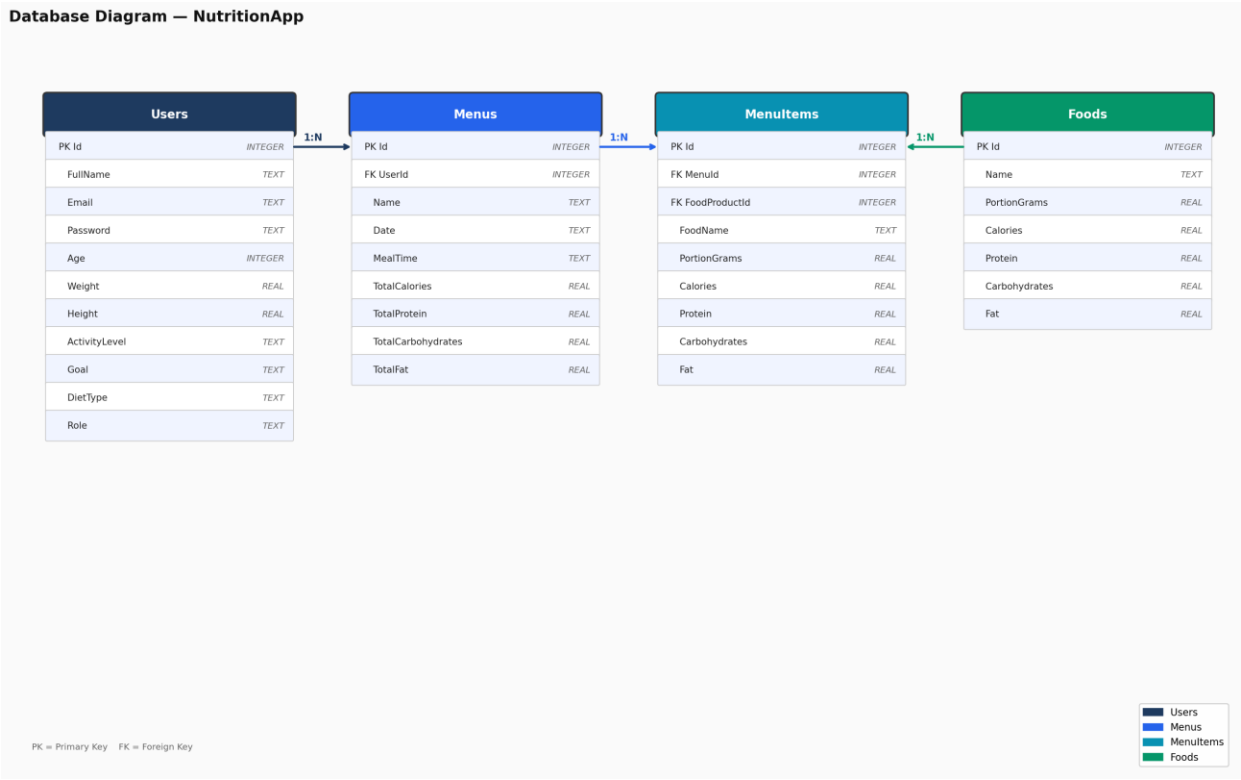
IsActive = true	Usuario activo, puede hacer login normalmente.
IsActive = false	Usuario desactivado, no puede hacer login. Sus datos se conservan.

2.8 Reportes CSV y PDF

Se implemento la descarga de reportes directamente desde el navegador usando IJSRuntime para invocar una función JavaScript que genera el archivo y lo descarga sin necesidad de guardar en el servidor. El PDF se genera con iText7 en memoria y se convierte a base64 para la descarga.

3. Diagrama de Base de Datos

La base de datos de NutritionApp está compuesta por cuatro tablas relacionadas entre sí. El diagrama muestra las tablas, sus campos, tipos de datos, claves primarias y foráneas, y las relaciones entre ellas.



3.1 Descripción de tablas

Tabla	Descripción	Relaciones
Users	Almacena todos los usuarios del sistema con su perfil nutricional y rol.	1:N con Menus
Menus	Almacena los menús diarios de cada usuario.	N:1 con Users, 1:N con MenuItems
MenuItems	Almacena los alimentos dentro de cada menú.	N:1 con Menus, N:1 con Foods
Foods	Catalogo de 250 alimentos con sus valores nutricionales.	1:N con MenuItems

4. Desarrollo

4.1 Estructura del proyecto

Carpeta/Proyecto	Contenido
NutritionApp/Models/	Clases de datos: User, Menu, MenuItem, FoodProduct, NutritionPlan y subclases, MacroDistribution.
NutritionApp/Controllers/	Logica de negocio: UserController, MenuController, FoodController, SuggestedMenuController.
NutritionApp/Data/	Repositorios, AppDbContext, DbInitializer y DataPaths.
NutritionApp/Services/	NutritionService para calculos nutricionales y SessionService para la sesion del usuario.
WebVisor/Components/Pages/	Paginas Blazor: Home, Dashboard, Menus, EditMenu, SuggestedMenus, Statistics, EditProfile, Reports, Admin, AdminUsers, AdminProducts, AdminStats.
Test/NutritionApp.Tests/	Pruebas unitarias MSTest para los 4 controladores.

4.2 Requisitos para ejecutar el programa

Sistema operativo	Windows 10 o superior
.NET SDK	.NET 10.0 o superior
IDE recomendado	Visual Studio Code o Visual Studio Community 2022
Base de datos	Se crea automáticamente al iniciar la aplicación

4.3 Como ejecutar el programa

Desde la terminal, en la carpeta raíz del proyecto:

```
dotnet run --project WebVisor/WebVisor.csproj
```

La aplicación abre en el navegador. La base de datos SQLite se crea automáticamente en la primera ejecución y se cargan los datos de los archivos JSON.

4.4 Credenciales de prueba

Usuario estándar	john@email.com / 1234
Administrador	admin@email.com / admin1234

4.5 Flujo de la aplicación

Pagina	URL	Rol	Descripción
Home / Login	/	Todos	Pantalla de inicio con formulario de login.
Dashboard	/dashboard	User	Información nutricional calculada automáticamente.
My Menus	/menus	User	Lista paginada de menus con crear, editar y eliminar.
Edit Menu	/editmenu/{id}	User	Formulario para crear o editar un menú.
Suggested Menus	/suggestedmenus	User	Menus predefinidos según tipo de dieta.
Statistics	/statistics	User	Estadísticas por rango de fechas con OK/MISS.
Edit Profile	/editprofile	User	Formulario para editar datos del perfil.
Reports	/reports	User	Descarga de reportes en CSV y PDF.
Admin Panel	/admin	Admin	Panel principal del administrador.
Manage Users	/adminusers	Admin	Crear, eliminar, resetear contraseñas, activar y desactivar usuarios.
Manage Products	/adminproducts	Admin	Agregar y eliminar productos del catálogo.
Global Statistics	/adminstats	Admin	Estadísticas globales de todos los usuarios.

5. Análisis de Resultados, Aprendizajes y Conclusiones

5.1 Análisis de resultados

Requerimiento	Estado	Observaciones
Migracion a Blazor Server	Completado	Todas las vistas reescritas en componentes. razor
Base de datos SQLite	Completado	Entity Framework Core con DbInitializer para datos iniciales
Rol de administrador	Completado	Panel admin con gestión de usuarios, productos, estadísticas globales, activar y desactivar usuarios
Reportes CSV y PDF	Completado	Descarga directa desde el navegador con iText7
Testing 80%+ cobertura	Completado	100% de cobertura en los 4 controladores con MSTest
Herencia y polimorfismo	Completado	NutritionPlan con 3 subclases mantenido desde v1
Patron Repository	Completado	Facilito la migración de JSON a SQLite sin cambiar controladores
Manejo de excepciones	Completado	Try/catch en repositorios y controladores

5.2 Conclusiones

Al finalizar el proyecto se logró construir una aplicación web funcional que permite a los usuarios registrar sus menús diarios, calcular sus calorías y macronutrientes según su perfil físico y sus objetivos, consultar estadísticas de su alimentación y descargar reportes. Además cuenta con un panel de administrador completo para gestionar usuarios y productos, y ofrece menús sugeridos según el tipo de dieta de cada usuario.

Uno de los retos más grandes fue migrar la aplicación a la web y lograr que todo funcionara junto de forma ordenada. Cuando llegó el momento de agregar nuevas funcionalidades como la base de datos, el panel de administrador y los reportes, no fue necesario reescribir nada, sino que la estructura ya estaba lista para recibir esos cambios. Eso me enseñó que pensar bien antes de actuar vale mucho más que actuar rápido.

Trabajar con una metodología organizada también cambió mi forma de ver los proyectos. Tener objetivos claros cada semana hizo que algo que al inicio se sentía enorme se volviera manejable y mucho menos abrumador.

NutritionApp no es perfecta, pero está bien construida, funciona correctamente y tiene una estructura clara. Como estudiante, lograr eso en un proyecto de esta escala es exactamente lo que se busca lograr.